

GoldenSetAuditor

Interview Defense Document v3.0

FAANG-Standard Complete Edition

Evaluation Dataset Quality Auditing for LLM and RAG Systems

Sidharth Kriplani | github.com/SidharthKriplani/goldensetauditor | PyPI: goldensetauditor

Label	Meaning
[IMPLEMENTED]	Code exists, runs, produces verified artifact
[SIMULATED]	Deliberately synthetic scenario; simulation logic is real code
[PARTIALLY VERIFIED]	Code exists but verification is internal consistency only
[FUTURE]	Not built — explicitly absent from the repo

QUICK REFERENCE

Metric	Value
Total checks	7
Checks that can FAIL	1 — conflicting labels only
Checks that WARN	5
Checks with INSUFFICIENT_INPUT	2 — near-dup at n>5K; coverage if no category_col
PyPI package	pip install goldensetauditor
Test cases	29
Output formats	JSON / Markdown / HTML
Current version	v0.5.0

Metric	Value
Core entry point	audit_golden_set(df, config)
Near-dup complexity	$O(n^2)$; sampled at $n > 5,000$
Runtime deps	pandas, numpy (no ML frameworks required)

STOC Table of Contents

- S0** Cover + Quick Reference
 - S1** The Problem — First Principles
 - S2** Architecture + All 7 Checks
 - S3** Algorithm Explanations with Worked Numerical Examples
 - S4** Design Decisions (6 explicit decisions, alternatives rejected)
 - S5** Honest Positioning — IMPLEMENTED / SIMULATED / FUTURE
 - S6** Q&A; Battery — 32 Pairs (FAANG Standard)
 - S7** Failure Modes
 - S8** Scalability
 - S9** Cross-Project Connections
 - S10** Resume Claim Audit
 - S11** Study Plan + Do Not Say This
-

How to Use This Document

This document is structured for deliberate, layered interview preparation. S1-S5 build the technical foundation — read these first to understand the project deeply enough to derive answers rather than recite them. S6 is the Q&A; battery — use it as a practice script, not a memorisation target. S7-S9 cover the adversarial questions that distinguish a candidate who built the tool from one who read about it. S10-S11 are the preparation scaffolding.

The target interview level is E5 (senior engineer) at FAANG. At this level, the interviewer expects you to: (1) explain WHY decisions were made, not just WHAT they do, (2) identify limitations proactively before being asked, (3) propose extensions grounded in trade-off analysis rather than hand-waving, and (4) connect individual components to system-level concerns. Every answer in S6 is written to satisfy these four criteria.

Before your interview: read S3 (algorithms) until you can reproduce every formula and worked example from memory. The formulas are simple arithmetic — token Jaccard, Shannon entropy, normalisation pipeline. If you can derive them live on a whiteboard, you will never be caught off guard by 'walk me through the math.'

S1 The Problem — First Principles

1.1 The Circularity No One Talks About

Every LLM and RAG evaluation workflow has a golden set at its centre: a collection of (prompt, expected_answer) pairs that serves as ground truth. When you benchmark a model you are asking how often the model output matches the expected answer. The implicit assumption is that the golden set is correct. But that assumption is never validated. You are using the golden set to validate the model, but nothing is validating the golden set. The tool that needs quality assurance is the one everyone assumes is already correct.

A bad golden set does not produce obviously wrong benchmark numbers — it produces confidently wrong ones. You do not discover the problem until a supposedly better model regresses on a category, or until two domain experts disagree on whether an expected answer was even correct. By then, months of model development decisions rest on a flawed foundation.

1.2 How Golden Sets Break in Practice

Golden sets are assembled by humans under deadline pressure. The failure modes are predictable:

Conflicting labels: Same question, two different expected answers. Happens when annotators work independently or when a set is updated without deduplication. The model's score on that question is undefined.

Exact duplicates: Same (prompt, answer) pair appears multiple times. Inflates example counts; a 100-example set with 30 duplicates is effectively 70 examples.

Weak expected answers: Single-word or trivially short references (Yes, No, 2). Creates binary-correct evaluations that do not measure model capability.

Ambiguous prompts: Prompts with unresolved pronouns or multiple embedded questions. No single correct answer exists.

Over-easy examples: The expected answer is extractable directly from prompt text. Measures copying, not reasoning.

Near-duplicate prompts: Paraphrased versions of the same question. A model that memorises one phrasing scores well on all paraphrases, inflating performance.

Coverage gaps: Some categories have only 2-3 examples. Per-category metrics on 2 examples have infinite variance.

1.3 Why Automated Auditing Instead of Manual Review

Manual review is inconsistent and undocumented. Two reviewers catch different issues. The same reviewer catches different issues on different days. Automated auditing produces structured, row-level evidence: 'prompt at row 47 is an exact duplicate of row 12' or 'category billing has only 2 examples, below minimum of 5.' That evidence is attachable to a model card, reviewable in a PR, and reproducible on every golden set update.

1.4 What GoldenSetAuditor Does NOT Do

Truth Boundary (printed on every report, cannot be suppressed): GoldenSetAuditor audits dataset quality. It does NOT evaluate model answers. It does NOT check whether expected answers are factually correct, whether the evaluation metric is appropriate, whether the golden set covers the production query distribution, or whether pretraining contamination has occurred.

1.5 Why This Matters for FAANG Evaluation Pipelines

At scale, evaluation infrastructure is shared across multiple teams. A single contaminated golden set can skew model selection decisions affecting millions of users. The cost of fixing a bad golden set discovered after 3 months of model development is orders of magnitude higher than running a 5-second automated audit at every golden set update. GoldenSetAuditor makes the audit cost negligible — it is a CI gate, not a manual process.

The industry trend toward automated LLM evaluation (MT-Bench, AlpacaEval, HELM) has focused on evaluation metric sophistication while leaving dataset quality mostly unaddressed. A gold-standard LLM judge scoring against a golden set with near-duplicate prompts produces inflated scores just as a simple exact-match evaluation would. The problem is the substrate, not the metric.

1.6 The Gap Between Academic Benchmarks and Production Evaluation Sets

Published academic benchmarks (MMLU, TriviaQA, HellaSwag) are curated by research teams with multi-round annotation review, IAA measurement, and adversarial filtering. They are not the golden sets GoldenSetAuditor is designed for.

Production RAG and LLM evaluation sets are different in every important way:

Origin: Assembled by product teams, support engineers, domain experts under deadline pressure — not professional annotators with IAA protocols.

Size: 50-2,000 examples, not hundreds of thousands. Statistical variance is higher; every quality issue has disproportionate impact on benchmark numbers.

Update frequency: Updated whenever product requirements change, product categories expand, or customer feedback identifies evaluation gaps. Academic benchmarks are mostly static; production sets change monthly.

Review process: Often a single domain expert who is also the person who created the golden set — no independent adversarial review.

Traceability: Version control often ad-hoc (CSV files in a shared drive) rather than git-managed. Conflict history is invisible.

GoldenSetAuditor is designed for this production context, not for re-auditing already-reviewed academic benchmarks. The tool's existence is justified by the systematic quality gap between how production evaluation sets are assembled and how they need to behave to produce reliable model development decisions.

S2 Architecture + All 7 Checks

2.1 Entry Point and Data Flow

The public API is a single function: `audit_golden_set(df, config)`. It accepts any pandas DataFrame and a `GoldenSetAuditConfig` dataclass. The config maps column names to their semantic roles and sets all thresholds. The function runs all 7 checks, aggregates results under a priority hierarchy (FAIL > WARN > INSUFFICIENT_INPUT > PASS), and returns a `GoldenSetReport` with per-finding evidence.

```
from goldensetauditor import GoldenSetAuditConfig, audit_golden_set config =
GoldenSetAuditConfig( input_col="question", expected_col="expected_answer",
category_col="category", # optional id_col="id", # optional min_expected_words=4,
near_duplicate_threshold=0.82, overeasy_overlap_threshold=0.72, min_category_count=3, )
report = audit_golden_set(df, config) print(report.status) # FAIL / WARN / INSUFFICIENT_INPUT
/ PASS report.save("outputs/") # writes JSON, Markdown, HTML
```

2.2 Check Architecture Overview

Group	Check	Method	Max Status
Structural	Conflicting labels	Exact match on normalised prompt; answers differ	FAIL
Structural	Exact duplicates	Exact match on normalised prompt; same answer	WARN
Content	Weak expected answer	Meaningful token count after stopword filter	WARN
Content	Ambiguous prompt	Anaphoric pronoun + multi-question-mark heuristic	WARN
Content	Over-easy example	Token Jaccard(prompt, answer) >= threshold	WARN

Group	Check	Method	Max Status
Content	Near-duplicate scan	Pairwise token Jaccard $O(n^2)$; sampled at $n > 5K$	WARN/INSUF
Coverage	Category coverage	Example count per category value	WARN/INSUF

2.2a GoldenSetAuditConfig — All Configurable Parameters

```
@dataclass class GoldenSetAuditConfig: input_col: str = 'prompt' expected_col: str =
'expected_answer' category_col: Optional[str] = None # if None, coverage -> INSUFFICIENT_INPUT
id_col: Optional[str] = None # for row identification in findings near_duplicate_threshold:
float = 0.82 # Jaccard >= this -> near-dup overeasiness overlap_threshold: float = 0.72 #
Jaccard(prompt,answer) -> over-easy min_expected_words: int = 4 # meaningful tokens after
stopword removal min_category_count: int = 3 # min examples per category max_pairwise_checks:
int = 25000 # sampling cap at n > 5000 diversity_entropy_warn: float = 0.55 # normalised H
below this -> WARN diversity_ttr_warn: float = 0.30 # type-token ratio below this -> WARN
```

All parameters have production-appropriate defaults. Domain teams override specific thresholds without touching code. A medical QA team would set `min_expected_words=2` (correct answers like 'Metformin' are single meaningful tokens). A conversational chatbot team might raise `near_duplicate_threshold=0.90` to tolerate template-similar prompts.

2.2b GoldenSetReport — Output Object Structure

```
audit_golden_set() returns a GoldenSetReport dataclass: @dataclass class GoldenSetReport:
overall_status: str # 'FAIL' | 'WARN' | 'INSUFFICIENT_INPUT' | 'PASS' findings:
List[GoldenSetFinding] # one per flagged issue audit_timestamp: str total_rows: int
config_used: GoldenSetAuditConfig truth_boundary: str # non-suppressible text @dataclass class
GoldenSetFinding: check_name: str # e.g. 'conflicting_labels' status: str # 'FAIL' | 'WARN' |
'INSUFFICIENT_INPUT' row_ids: List # affected row identifiers message: str # human-readable
description evidence: dict # check-specific structured evidence
```

2.3 Conflicting Labels [IMPLEMENTED — Only FAIL check]

Two rows with identical normalised prompt text but different expected answers represent a structural impossibility in evaluation. If the model produces answer A, is it correct or incorrect? The ground truth is undefined. This is the only FAIL-level check because it is the only condition that makes the evaluation logically broken, not just weaker. Normalisation pipeline: lowercase, strip whitespace, collapse multiple spaces, remove leading/trailing punctuation.

2.4 Exact Duplicates [IMPLEMENTED]

Same prompt, same expected answer. Redundant examples inflate the count without adding evaluation coverage. WARN not FAIL because duplicates do not break the evaluation contract — they

just waste evaluation budget.

2.5 Weak Expected Answer [IMPLEMENTED]

Expected answers with fewer than `min_expected_words` (default 4) meaningful tokens after stopword filtering are flagged WARN. The stopword set removes articles, prepositions, common verbs, and filler words. An expected answer of 'the answer is yes' contains 1 meaningful token, not 4. Single-token answers create trivially-correct or trivially-incorrect evaluations that do not measure model capability.

2.6 Ambiguous Prompt [IMPLEMENTED]

Two patterns: (1) Anaphoric references — prompts containing it/they/this/that in short prompts (<50 tokens) where the referent is not established in the prompt text. (2) Multiple question marks — prompts with more than one ? ask multiple questions but are evaluated against a single expected answer.

2.7 Over-Easy Example [IMPLEMENTED]

Token Jaccard between prompt and expected answer measures whether the model could answer by copying from the prompt. $\text{Jaccard} = |\text{tokens}(\text{prompt}) \cap \text{tokens}(\text{answer})| / |\text{tokens}(\text{prompt}) \cup \text{tokens}(\text{answer})|$. Threshold 0.72 by default. Over-easy examples test extraction, not reasoning.

2.8 Near-Duplicate Scan [IMPLEMENTED]

Pairwise token Jaccard computed across all prompt pairs. Pairs with Jaccard $\geq$ `near_duplicate_threshold` (default 0.82) are flagged. Complexity $O(n^2)$. Above $n=5,000$ the auditor samples `max_pairwise_checks=25,000` pairs randomly and returns `INSUFFICIENT_INPUT` with a note.

2.9 Category Coverage [IMPLEMENTED]

For each unique value in `category_col`, verifies at least `min_category_count` (default 3) examples. A category with 2 examples achieving 100% model accuracy is statistically meaningless. If no `category_col` is provided: `INSUFFICIENT_INPUT`, not PASS.

2.10 How All 7 Checks Interact — Priority Aggregation Logic

The seven checks run independently. Each returns a list of `GoldenSetFinding` objects. The aggregation function then determines the overall_status:

```
def aggregate_status(findings: List[GoldenSetFinding]) -> str: statuses = {f.status for f in findings} if 'FAIL' in statuses: return 'FAIL' if 'WARN' in statuses: return 'WARN' if 'INSUFFICIENT_INPUT' in statuses: return 'INSUFFICIENT_INPUT' return 'PASS'
```


This is a straightforward priority scan, not a weighted score. The design decision is intentional: no finding cancels another. If five checks pass and one produces WARN, the overall status is WARN. If any check produces FAIL, the overall status is FAIL regardless of how many other checks passed. This is conservative by design — a golden set with any FAIL requires correction before evaluation results are trustworthy.

2.11 The STOPWORDS Set — Design and Justification

The STOPWORDS set is hardcoded in core.py. Current contents:

```
STOPWORDS = { # Articles 'the', 'a', 'an', # Prepositions 'to', 'of', 'for', 'in', 'on',  
'with', 'at', 'by', 'from', 'as', 'into', # Common verbs 'is', 'are', 'was', 'were', 'does',  
'do', 'did', 'can', 'could', 'should', 'would', 'will', 'has', 'have', 'had', 'be', 'been', #  
Question words 'what', 'how', 'why', 'when', 'where', 'which', 'who', # Filler 'please',  
'explain', 'tell', 'me', 'about', 'this', 'that', 'it', 'they', 'their', 'its', 'our', 'your',  
'my', # Conjunctions 'and', 'or', 'but', 'if', 'then', 'because', 'so', 'yet', # Numbers /  
tokens that add noise 'not', 'no', 'yes', 'also', 'just', 'more', 'most', 'some', }
```

The STOPWORDS set is not configurable in the current version — it is a fixed English vocabulary. This is a known limitation for non-English golden sets and for domain vocabularies where words like 'not' are semantically critical (e.g., medical QA where 'does not' vs 'does' is the entire answer distinction). For v1.0, STOPWORDS would become a config parameter with the current set as default.

2.12 Test Suite — How the 29 Tests Are Structured

The 29 tests follow a consistent pattern. For each check there are at minimum: (1) a positive test — data that must trigger the finding, (2) a negative test — clean data that must NOT trigger the finding, and (3) one or more edge cases.

Test Category	Count	What It Verifies
Conflicting labels — positive	2	Single conflict triggers FAIL; multiple conflicts all reported
Conflicting labels — negative	1	Clean set with unique prompts returns no FAIL finding
Exact duplicates — positive	2	Full duplicate pair triggers WARN; partial duplicate (same prompt, diff answer) is caught by conflicting labels instead
Weak answer — positive	2	Single-token answer and empty answer both trigger WARN
Weak answer — negative	1	Answer with 4+ meaningful tokens passes
Ambiguous prompt — positive	2	Anaphoric pronoun in short prompt; multiple question marks
Over-easy — positive	2	High-overlap prompt-answer pair; Jaccard at exact threshold boundary
Near-dup — positive	2	Jaccard just above threshold triggers; Jaccard just below does not

Test Category	Count	What It Verifies
Near-dup — sampling	1	At n>5000 the check returns INSUFFICIENT_INPUT not WARN
Category coverage — positive	2	Category with 2 examples triggers WARN; category with 3 passes
Category coverage — INSUFFICIENT_INPUT	1	No category_col provided returns INSUFFICIENT_INPUT, not PASS
Full audit PASS	2	A carefully constructed clean 20-row set returns overall PASS
Status hierarchy	2	FAIL dominates WARN; WARN dominates INSUFFICIENT_INPUT
Output format	3	JSON, Markdown, HTML all produced by report.save()
Config defaults	2	Default config works without optional parameters

The test suite does not include integration tests against real-world golden sets from published benchmarks (MMLU, TriviaQA). This is a documented gap — those tests would require licensing decisions and human expert labeling of true positive findings. The current 29 tests are sufficient to maintain correctness across code changes but are not sufficient to claim measured precision/recall.

S3 Algorithm Explanations with Worked Numerical Examples

3.1 Token Jaccard — The Core Primitive

Token Jaccard is the mathematical foundation of three checks: over-easy detection, near-duplicate scanning, and implicitly the answer diversity analysis. Formula: $J(A, B) = |\text{tokens}(A) \cap \text{tokens}(B)| / |\text{tokens}(A) \cup \text{tokens}(B)|$. Tokens are extracted via regex `[a-zA-Z0-9]+`, lowercased, stopwords removed, single-character tokens removed.

Worked Example 1 — Over-Easy Detection:

```
Prompt: 'What is the capital of France and what is its population?' Answer: 'The capital is France Paris with population 2 million'
prompt_tokens (after stopword removal): {capital, france, population}
answer_tokens (after stopword removal): {capital, france, paris, population, million}
Intersection: {capital, france, population} = 3 tokens
Union: {capital, france, paris, population, million} = 5 tokens
Jaccard = 3/5 = 0.60 -- below threshold 0.72, NOT flagged.
Now: Answer = 'capital france population'
answer_tokens = {capital, france, population}
Jaccard = 3/3 = 1.0 -- FLAGGED as over-easy.
```

Worked Example 2 — Near-Duplicate Detection:

```
Prompt A: 'How do I reset my password?' Prompt B: 'What are the steps to reset my password?'
tokens(A) = {reset, password} tokens(B) = {steps, reset, password}
Intersection: {reset, password} = 2
Union: {steps, reset, password} = 3
Jaccard = 2/3 = 0.667 -- below threshold
```

0.82, NOT flagged. The 0.82 threshold is deliberately conservative. Two prompts must share most of their meaningful vocabulary to be near-duplicates.

3.2 Near-Duplicate Complexity and the Sampling Decision

For n examples, pairwise comparison produces $n*(n-1)/2$ pairs:

$n=100 \rightarrow 4,950$ pairs (< 1 second) $n=1,000 \rightarrow 499,500$ pairs (few seconds) $n=5,000 \rightarrow 12,497,500$ pairs (minutes -- sampled instead) $n=10,000 \rightarrow 49,995,000$ pairs (unacceptable in a CI gate)

Above $n=5,000$: 25,000 random pairs are sampled and the check returns `INSUFFICIENT_INPUT` with a note. A sampled check with declared limitations is more honest than a full check that times out or silently skips.

3.3 Shannon Entropy and Answer Diversity (v0.5.0)

The diversity check computes three metrics over all expected answers collectively: (1) Normalised Shannon entropy of token frequency distribution; (2) Type-Token Ratio (`unique_tokens / total_tokens`); (3) Coefficient of variation of answer lengths.

Worked Example — Shannon Entropy:

Answers: ['yes', 'yes', 'yes', 'no', 'yes'] Token counts: {'yes': 4, 'no': 1}, total = 5
 $p('yes') = 0.8$, $p('no') = 0.2$ $H = -(0.8 * \log_2(0.8) + 0.2 * \log_2(0.2)) = -(0.8 * (-0.322) + 0.2 * (-2.322)) = 0.722$ bits Normalised by $\log_2(2) = 1.0 \rightarrow H_{\text{norm}} = 0.722$ With
`diversity_entropy_warn=0.55`: $0.722 > 0.55 \rightarrow$ no warning. If all 5 answers were 'yes': $H = 0.0 \rightarrow$ WARN fires (homogeneous vocabulary).

3.4 Conflicting Labels — Normalisation Pipeline

Conflicting label detection requires a normalisation step to prevent trivial presentation differences from causing missed detections. A prompt annotated as 'What is Python?' by one annotator and 'what is python ' (trailing space, no question mark) by another are semantically identical and must be matched.

```
def normalise(text): text = text.lower() # case-fold text = text.strip() # leading/trailing
whitespace text = re.sub(r'\s+', ' ', text) # collapse interior spaces text =
re.sub(r'^[^\w]+|[\w]+$', '', text) # strip boundary punctuation return text Examples: 'What
is Python?' -> 'what is python' 'What is Python ' -> 'what is python' (match: same conflict)
'WHAT IS PYTHON' -> 'what is python' (match: same conflict) 'What is Python?!' -> 'what is
python' (match: same conflict) 'What is Python 3?' -> 'what is python 3' (different: '3' is
content word)
```

What the normalisation does NOT do: Unicode normalisation (NFC/NFKC), punctuation removal within text (only at boundaries), stemming or lemmatisation. The decision not to normalise Unicode is a known gap — accented characters (e.g., 'café' vs 'cafe') would not match. NFKC normalisation is tracked as a v0.6.0 fix.

3.5 Complete End-to-End Audit Walkthrough

Suppose we have this 6-row golden set:

```
id | prompt | expected_answer | category
---+-----+-----+----- 1 | What is
the capital of France? | Paris is the capital. | geography 2 | What is the capital of France?
| The capital is Lyon. | geography 3 | What is the capital of Germany? | Berlin | geography 4
| What are the steps to reset pw? | reset password steps | support 5 | How do I reset my
password? | reset password steps | support 6 | What is a binary tree? | A binary tree is a...
| cs
```

Running `audit_golden_set()` on this set produces: Check 1 – Conflicting labels: rows 1 and 2 have identical normalised prompt 'what is the capital of france' but expected answers 'paris is the capital' vs 'the capital is lyon'. Finding: FAIL. `overall_status` = FAIL immediately. Check 2 – Exact duplicates: no exact (prompt+answer) pairs. No finding. Check 3 – Weak expected answer: row 3 answer 'Berlin' has 1 meaningful token (below default `min_expected_words=4`). Finding: WARN on row 3. Check 4 – Ambiguous prompt: no unresolved pronouns or multiple question marks. No finding. Check 5 – Over-easy: row 4 answer 'reset password steps' vs prompt 'steps to reset pw'. `prompt_tokens` = {steps, reset} (removing 'to', 'pw' is kept as content word) `answer_tokens` = {reset, password, steps} Jaccard = $2/4 = 0.50$ -- below 0.72, not flagged. Check 6 – Near-duplicate: rows 4 and 5: `tokens(4)` = {steps, reset, password} `tokens(5)` = {reset, password} Intersection: {reset, password} = 2 Union: {steps, reset, password} = 3 Jaccard = $2/3 = 0.667$ -- below 0.82, NOT flagged. Would need Jaccard ≥ 0.82 . Check 7 – Category coverage: `geography=3`, `support=2`, `cs=1`. `support` ($2 < 3$) -> WARN. `cs` ($1 < 3$) -> WARN. Final status: FAIL (conflicting labels dominates). Findings: 1 FAIL (rows 1,2), 1 WARN (row 3), 2 WARN (categories support, cs).

3.6 Type-Token Ratio (TTR) and Answer Diversity

The answer diversity check computes three complementary metrics. Understanding all three prevents over-reliance on any single signal:

Metric 1: Normalised Shannon Entropy

$H_{\text{norm}} = H / \log_2(\text{vocab_size})$ where $H = -\sum(p(t) * \log_2(p(t)))$ for all tokens t $H_{\text{norm}} = 0.0$ means all expected answers contain the same single token. $H_{\text{norm}} = 1.0$ means every token appears exactly once (maximum diversity). Warn threshold: $H_{\text{norm}} < 0.55$ (default `diversity_entropy_warn`).

Metric 2: Type-Token Ratio (TTR)

$TTR = \text{unique_token_count} / \text{total_token_count}$ Example: 5 answers with tokens ['yes', 'yes', 'yes', 'no', 'yes'] `unique_tokens` = 2 ('yes', 'no'), `total_tokens` = 5 $TTR = 2/5 = 0.40$ Warn threshold: $TTR < 0.30$ (default `diversity_ttr_warn`). Interpretation: a golden set where 70%+ of tokens are repeats may have templated expected answers that test label matching rather than model capability.

Metric 3: Coefficient of Variation of Answer Lengths

$CV = \text{std}(\text{answer_lengths}) / \text{mean}(\text{answer_lengths})$ A golden set where every expected answer is 5-7 tokens long has CV near 0. This indicates a templated format rather than naturally varied reference answers. High CV (> 1.0) indicates wide variance in expected answer length, which

can indicate the golden set spans multiple question types (desirable) or inconsistent annotation standards (undesirable – requires human review).

The three metrics together catch different failure modes: entropy catches vocabulary concentration, TTR catches template reuse, CV catches length homogeneity. A golden set that passes all three is likely to have meaningfully diverse reference answers across its full question distribution.

3.7 The Check Pipeline — Implementation Internals

Understanding the implementation at source-code level is necessary for interview questions like 'walk me through your codebase' or 'how would you add a new check'. The key pattern is that every check is a pure function with this exact signature:

```
def check_something( df: pd.DataFrame, config: GoldenSetAuditConfig ) ->
List[GoldenSetFinding]: """Returns empty list if no issues found.""" findings = [] # ... check
logic ... return findings
```

This signature has three important properties: (1) Pure function — no side effects, no global state mutation, same inputs always produce same outputs. This makes each check independently testable without mocking. (2) Returns empty list on PASS — the caller aggregates findings from all checks; an empty list means nothing to report for that check. (3) Returns List, not a single finding — one check might flag multiple rows (e.g., near-dup might find 5 pairs; conflicting labels might find 3 conflicts).

The conflicting labels check implementation pattern:

```
def check_conflicting_labels(df, config): findings = [] # normalise prompts norm =
df[config.input_col].apply(normalise) # group by normalised prompt, find groups with > 1
unique answer groups = df.groupby(norm)[config.expected_col].apply(set) conflicts =
groups[groups.apply(len) > 1] for prompt, answers in conflicts.items(): # find all row IDs
with this prompt mask = norm == prompt row_ids = df[mask][config.id_col].tolist() if
config.id_col else \ df[mask].index.tolist() findings.append(GoldenSetFinding(
check_name='conflicting_labels', status='FAIL', row_ids=row_ids, message=f'Prompt appears
with {len(answers)} different expected answers', evidence={'prompt': prompt, 'answers':
list(answers)} )) return findings
```

The near-duplicate scan is the most complex check due to $O(n^2)$ pairwise logic:

```
def check_near_duplicates(df, config): findings = [] prompts = df[config.input_col].tolist() n
= len(prompts) token_sets = [_tokens(p) for p in prompts] # stopword-filtered token sets
pairs_to_check = [] if n * (n-1) // 2 > config.max_pairwise_checks: # Sample
max_pairwise_checks random pairs import random all_pairs = [(i,j) for i in range(n) for j in
range(i+1, n)] pairs_to_check = random.sample(all_pairs, config.max_pairwise_checks) sampled
= True else: pairs_to_check = [(i,j) for i in range(n) for j in range(i+1, n)] sampled = False
for i, j in pairs_to_check: jaccard = _jaccard(token_sets[i], token_sets[j]) if jaccard >=
config.near_duplicate_threshold: findings.append(GoldenSetFinding(
check_name='near_duplicates', status='INSUFFICIENT_INPUT' if sampled else 'WARN', row_ids=[i,
j], message=f'Jaccard={jaccard:.3f} >= threshold {config.near_duplicate_threshold}',
evidence={'jaccard': jaccard, 'sampled': sampled} )) return findings
```

The `_jaccard()` helper is the atomic unit called by three different checks:

```
def _jaccard(set_a: set, set_b: set) -> float: if not set_a and not set_b: return 0.0 # two empty prompts are not near-duplicates by convention intersection = len(set_a & set_b) union = len(set_a | set_b) return intersection / union if union > 0 else 0.0
```

The empty set edge case (return 0.0 for two empty prompts) is important: an empty-string prompt pair should not be flagged as near-duplicates because the check would produce a meaningless finding. They will instead be caught by the weak-answer check and potentially the conflicting-labels check.

S4 Design Decisions — 6 Explicit Choices, Alternatives Rejected

4.1 FAIL Only for Conflicting Labels

Decision: Every other check reduces evaluation informativeness without making it structurally invalid. FAIL is reserved for structural invalidity: the ground truth is undefined when the same prompt has two different correct answers. Everything else is WARN because it requires human judgment to decide whether it matters for that specific domain.

Alternative rejected: Alternative rejected: FAIL for near-duplicates. Near-duplicates are a coverage problem, not a logical contradiction. Two similar prompts might coexist intentionally to test paraphrase robustness. A FAIL would block valid golden sets.

4.2 Token Jaccard Instead of Embedding Similarity for Near-Dup

Decision: Token Jaccard requires zero external ML dependencies, runs in pure Python, produces deterministic results (same input always produces same output), and is explainable to any reviewer. For most golden sets assembled by one team, surface-form near-duplicates are the dominant failure mode.

Alternative rejected: Alternative rejected: sentence-transformers cosine similarity. Adds a GPU/CPU dependency, model versioning concerns, and non-determinism across platforms. Tracked as a roadmap item for large golden sets where semantic similarity matters more.

4.3 INSUFFICIENT_INPUT as a Distinct Status

Decision: A check that cannot run because optional configuration was not provided must not return PASS. Returning PASS would silently skip checks, giving a false sense of completeness. INSUFFICIENT_INPUT makes the gap explicit — the caller knows exactly which checks were skipped

and why.

Alternative rejected: Alternative rejected: silently skip unavailable checks. Silent skips allow golden sets to accumulate coverage gaps while always showing PASS. The audit contract must be explicit about what was and was not checked.

4.4 Truth Boundary Cannot Be Suppressed

Decision: The truth boundary text is printed on every output format and cannot be removed via configuration. This prevents reports from being misused as evidence that a golden set is fully validated — which would require domain expert review beyond what automated checks can provide.

Alternative rejected: Alternative rejected: make the truth boundary configurable. A configurable truth boundary would eventually be turned off by teams under deadline pressure, exactly the condition where it is most important.

4.5 PyPI Publication and Zero-Configuration Entry Point

Decision: Publishing to PyPI means any team can audit their golden set with three lines of code. The core entry point (`audit_golden_set`) requires no subclassing or factory patterns. Evaluation quality tooling only works if it is actually used, and friction kills adoption.

Alternative rejected: Alternative rejected: require `GoldenSetAuditor(df, config).audit()` class pattern only. Accepted for flexibility (the class API exists) but the functional entry point was added for the 95% case where teams just need a report.

4.6 Configurable Thresholds with Conservative Defaults

Decision: All thresholds are configurable via `GoldenSetAuditConfig`. A medical QA golden set might need a lower near-duplicate threshold (0.6) because paraphrases matter; a chatbot golden set might need a lower weak-answer threshold (2 tokens) because short answers are domain-appropriate. One-size-fits-all thresholds would over-warn or under-warn.

Alternative rejected: Alternative rejected: fixed thresholds. Different domains have fundamentally different answer length distributions and acceptable near-duplicate rates.

4.7 Why the Status Hierarchy Has Four Levels, Not Two

Decision: `FAIL / WARN / INSUFFICIENT_INPUT / PASS` is a four-level hierarchy, not a binary pass/fail. This gives report consumers actionable signal: `FAIL` means stop and fix; `WARN` means review before trusting; `INSUFFICIENT_INPUT` means re-run with more config; `PASS` means proceed

with documented caveats.

Alternative rejected: Binary PASS/FAIL. A binary system would either block CI on minor issues (WARN as FAIL) causing alert fatigue, or silently hide them (WARN as PASS) removing all signal. The four-level hierarchy matches how experienced data engineers actually triage quality issues.

4.8 Structured Evidence in Every Finding, Not Just a Status String

Decision: Every GoldenSetFinding carries structured evidence: the affected row_ids, the actual values that triggered the check, and a human-readable message. Evidence is attached at the finding level, not summarised at the report level. This makes every finding independently actionable without re-running the audit.

Alternative rejected: Summary counts only ('3 near-duplicate pairs found'). A count without row IDs forces the data team to re-investigate what the auditor already identified. Structured evidence means 'row 47 and row 23 are near-duplicates with Jaccard=0.88 — here are the two prompts' — immediately actionable.

4.9 How Default Thresholds Were Chosen

Decision: Default thresholds are conservative — they prefer false positives (human review of acceptable data) over false negatives (silent pass on bad data). The calibration reasoning for each default:

near_duplicate_threshold = 0.82:

Token Jaccard of 0.82 means sets A and B share 82% of their union vocabulary. At this level, prompts are paraphrases of each other. Below 0.82, prompts share significant vocabulary but may cover meaningfully different aspects. Empirical intuition: 'How do I reset my password?' and 'What steps do I follow to reset my password?' have Jaccard ~ 0.67 (reset, password in both; steps in one). Threshold 0.82 is set above this to allow such legitimate variations. Teams with higher paraphrase tolerance can raise it to 0.90.

overeasy_overlap_threshold = 0.72:

Token Jaccard(prompt, answer) of 0.72 means 72% of the union vocabulary appears in both prompt and answer. At this level, answering the question requires no new information beyond what was already stated in the prompt. Example: prompt = 'The company was founded by Bezos in 1994. When was it founded?' has tokens {company, founded, bezos, 1994}; answer = '1994 bezos founded' has the same tokens. Jaccard = 1.0. Threshold 0.72 allows prompts that mention context while still requiring the model to identify the relevant piece.

min_expected_words = 4:

Four meaningful tokens is the minimum that constitutes a sentence-fragment answer. 'Paris capital France' is 3 tokens and is a label, not an explanation. 'Paris is the capital of France' after stopword removal is 3 tokens: {paris, capital, france}. Still borderline. Four tokens forces at least one additional content word beyond subject-verb-object: '{paris, capital, france, city}' or '{paris, capital, france, population}'. Teams evaluating yes/no questions should lower this to 1.

min_category_count = 3:

Two examples per category produce unreliable per-category accuracy estimates. With $n=2$, perfect accuracy and 50% accuracy are both plausible for any model. Three examples is the minimum for a meaningful majority vote. Five examples would be more reliable but would reject many valid production golden sets. Default 3 is the minimum useful signal.

Alternative rejected: Data-driven threshold calibration from a corpus of known-good golden sets. This would require collecting labeled golden sets with human expert annotations of true positive / false positive findings — a substantial annotation project that does not exist. The conservative manual defaults are preferable to uncalibrated data-driven thresholds.

S5 Honest Positioning — IMPLEMENTED / SIMULATED / FUTURE

Feature	Status	Evidence
Conflicting label detection	[IMPLEMENTED]	core.py; 29 tests pass
Exact duplicate detection	[IMPLEMENTED]	core.py check_exact_duplicates
Weak expected answer check	[IMPLEMENTED]	STOPWORDS set + token count
Ambiguous prompt detection	[IMPLEMENTED]	anaphora + multi-? heuristic
Over-easy example detection	[IMPLEMENTED]	token Jaccard(prompt, answer)
Near-duplicate scan (token Jaccard)	[IMPLEMENTED]	pairwise $O(n^2)$; sampled at $n>5K$
Category coverage check	[IMPLEMENTED]	per-category count vs min threshold
Answer diversity metrics (v0.5.0)	[IMPLEMENTED]	Shannon entropy + TTR + CV length
JSON/Markdown/HTML output	[IMPLEMENTED]	GoldenSetReport.save()
PyPI publication	[IMPLEMENTED]	pip install goldensetauditor works
29-test suite	[IMPLEMENTED]	tests/ directory in repo
Embedding-based semantic near-dup	[FUTURE]	Roadmap; not in v0.5.0
LLM-judge expected answer check	[FUTURE]	Would require external API

Feature	Status	Evidence
Pretraining contamination detection	[FUTURE]	Requires training corpus access
Multi-turn full context support	[PARTIALLY]	Works on final-turn; anaphora may FP

FAANG interview note: When asked what [IMPLEMENTED] means for real, the answer is: code exists in src/, a CI test covers it, the README demo produces real output, and the PyPI package installs and runs it. Not prototyped. Shipped.

5.2 Honest Self-Assessment of Limitations

The following limitations are known and documented, not hidden:

No ground-truth validation: GoldenSetAuditor cannot tell you if expected answers are factually correct. A golden set where every expected answer is plausible but wrong will PASS all checks.

English-centric: STOPWORDS list is English. Tokenisation regex `[a-zA-Z0-9]+` is Latin-script only. Running on Chinese, Arabic, or Japanese golden sets will produce meaningless token counts.

No production distribution alignment check: A golden set can pass all 7 checks while being completely unrepresentative of production query distribution. No automated check can catch this without access to production query logs.

Near-dup sampling at large n: Above 5,000 rows, only 25,000 random pairs are checked. The false-negative rate for near-duplicate detection increases with n. Documented in INSUFFICIENT_INPUT finding, not silently treated as PASS.

Heuristic ambiguity detection: The anaphora and multiple-question-mark heuristics have unmeasured false positive rates. A labeled evaluation dataset of ambiguity judgments from domain experts does not exist.

5.3 Version History — What Changed Between Releases

Understanding the evolution of the tool demonstrates sustained engineering thought:

Version	Key Changes	Design Rationale
v0.1.0	Conflicting labels + exact duplicate checks only	Smallest useful tool: the two checks that break evaluation contracts most severely.
v0.2.0	Added weak answer, ambiguous prompt, over-easy checks	Content quality checks — address the failure modes beyond structural invalidity.
v0.3.0	Added near-duplicate scan (token Jaccard $O(n^2)$ with sampling cap)	The most impactful check for manually assembled golden sets. Added last because it required the most careful complexity management.

Version	Key Changes	Design Rationale
v0.4.0	Added category coverage check; INSUFFICIENT_INPUT status introduced	Coverage gaps are invisible without category_col. INSUFFICIENT_INPUT needed to distinguish 'check could not run' from PASS.
v0.5.0	Added Shannon entropy / TTR / CV diversity check; HTML output format	Diversity metrics catch homogeneous answer distributions that pass all other checks. HTML report for human reviewers.
v1.0 (planned)	LLM-judge answer verification; semantic ANN near-dup; HELM/lm-eval hooks	Moves beyond heuristics to semantic understanding. Requires accepting external dependencies.

5.4 What [IMPLEMENTED] Means Under Scrutiny

An interviewer at FAANG will push on every [IMPLEMENTED] claim. The test is: can you demonstrate it, not just assert it. For GoldenSetAuditor, every [IMPLEMENTED] feature satisfies all four criteria:

- 1. Code exists in src/:** The check function is in core.py. You can point to the specific function by name (e.g., check_conflicting_labels, check_near_duplicates).
- 2. A CI test covers it:** At least one pytest case in tests/ exercises the check with a DataFrame that should trigger it, and asserts the correct status and finding. The test fails if the check is removed.
- 3. The README demo produces real output:** The usage example in README.md, when run, produces a GoldenSetReport with real findings. Not a mock. Not a screenshot. Runnable code with runnable output.
- 4. The PyPI package installs and runs it:** pip install goldensetauditor followed by the 3-line usage example produces a report. This is the ultimate integration test — the package is installed from PyPI, not from a local editable install.

If an interviewer asks 'show me', the answer to every [IMPLEMENTED] feature is: 'here is the function, here is the test, here is the output from pip install.'

5.5 Sample Audit Report Output — What the JSON Looks Like

Being able to describe the output format concretely is necessary for interview questions like 'how would you integrate this into a CI pipeline' or 'what does the report contain'. Below is a representative JSON output for the 6-row example from S3.5:

```
{ "overall_status": "FAIL", "total_rows": 6, "audit_timestamp": "2025-11-14T09:23:41Z",
"truth_boundary": "GoldenSetAuditor audits dataset quality only. It does NOT ...", "findings":
[ { "check_name": "conflicting_labels", "status": "FAIL", "row_ids": [1, 2], "message":
"Prompt appears with 2 different expected answers", "evidence": { "prompt": "what is the
capital of france", "answers": ["Paris is the capital.", "The capital is Lyon."] } }, {
"check_name": "weak_expected_answer", "status": "WARN", "row_ids": [3], "message": "Expected
answer has 1 meaningful token (< min 4)", "evidence": {"answer": "Berlin",
```

```
"meaningful_tokens": ["berlin"]}] } }, { "check_name": "category_coverage", "status": "WARN",  
"row_ids": [], "message": "Categories below min count (3): support=2, cs=1", "evidence":  
{"underpopulated": {"support": 2, "cs": 1}} } ] }
```

Key structural observations: (1) overall_status at top level for quick CI check, (2) truth_boundary non-suppressible, (3) each finding identifies the specific rows affected (row_ids = [1, 2] for the conflict), (4) evidence carries the actual values that triggered the finding — the PR reviewer sees the conflict without having to re-run the audit.

S6 Q&A; Battery — 32 Pairs (FAANG Standard)

This section contains 32 Q&A; pairs organised by question type. Questions 1-15 are the required FAANG-standard core questions — these should be answered without hesitation. Questions 16-32 are depth and stress-test questions that an E5/E6 interviewer might probe. For each answer, the key principle being tested is the WHY, not just the WHAT. If you can explain the reasoning behind every design decision, you can handle any paraphrase or follow-up the interviewer throws.

6.1 Architecture and System Design Questions

6.1 Core Architecture and Data Flow (Q1–Q5)

Q: 1. Walk me through what happens when audit_golden_set() receives a 500-row DataFrame.

The function first validates that required columns exist (input_col, expected_col). It then runs all 7 checks independently — each is a pure function returning a GoldenSetFinding list. Checks needing optional config (category_col for coverage, near-dup for large n) return INSUFFICIENT_INPUT if config is absent. After all checks run, the status is aggregated: FAIL if any finding is FAIL; else WARN if any WARN; else INSUFFICIENT_INPUT if any; else PASS. For 500 rows: conflicting labels $O(n)$, exact dup $O(n)$, content checks $O(n)$, near-dup $O(n^2) = 124,750$ pairs. Full audit completes in under 2 seconds.

Q: 2. Why does only conflicting labels produce FAIL and everything else produce WARN?

FAIL is reserved for structural invalidity. Conflicting labels make the evaluation contract logically undefined: if the same prompt has two different correct answers, no model output can be consistently classified as correct or incorrect. Every metric on that row is meaningless. Every other quality issue reduces informativeness or efficiency without making the evaluation structurally invalid. Weak answers, near-dups, ambiguous prompts are addressable: you can decide a short answer is domain-appropriate, or that two similar prompts are intentionally testing paraphrase robustness. You cannot decide that having two contradictory ground truths is acceptable.

Q: 3. How does your near-duplicate scan work mathematically? What is the complexity?

The near-dup scan computes token Jaccard for every pair of prompts. For each pair (A, B): $\text{tokens}(A) = \text{set of non-stopword tokens}$; $\text{Jaccard} = |A \cap B| / |A \cup B|$. If $\text{Jaccard} \geq \text{near_duplicate_threshold}$ (default 0.82), the pair is flagged. Complexity: $O(n^2)$ pairs, each comparison $O(k)$ where k is the token set size. For $n=500$: 124,750 pairs — sub-second. For $n=5,000$: 12.5M pairs — minutes. Above 5,000 the auditor samples 25,000 random pairs and returns INSUFFICIENT_INPUT. Sampling is honest: a declared limitation beats a false PASS.

Q: 4. Token Jaccard misses semantic duplicates — how would you extend to catch those?

Token Jaccard is a surface-form measure — it catches 'How do I reset my password?' vs 'What are the steps to reset my password?' but misses 'How do I recover account access?' which is semantically equivalent. Extension: encode all prompts with a sentence-transformer (e.g., all-MiniLM-L6-v2), compute cosine similarity between all pairs, flag pairs above a semantic threshold (e.g., 0.85). This is on the roadmap for $n > 2,000$ golden sets. Tradeoff: adds GPU/CPU dependency and model versioning concerns. Current design keeps it optional — if sentence-transformers is installed, semantic near-dup is enabled; otherwise falls back to token Jaccard.

Q: 5. What is your false positive rate on the ambiguous prompt check?

Not empirically measured — no labeled false-positive dataset exists. The check uses two heuristics: anaphoric pronoun detection (it/they/this/that in short prompts < 50 tokens) and multiple question marks. Both have known false positive cases. Anaphoric pronouns are common in multi-turn datasets where 'it' refers to a previously established entity. Multiple question marks can appear in prompts asking 'What is X and how does it work?' which is a single compound question, not ambiguous. Mitigation: the check produces WARN not FAIL, so false positives require human review but do not block the pipeline. The check can be disabled for multi-turn datasets.

6.2 Algorithm Design and Mathematical Foundations (Q6–Q10)

Q: 6. How do you define 'weak expected answer' — what is your stopwords list approach?

The check counts meaningful tokens in the expected answer after removing stopwords. The STOPWORDS set includes: articles (the, a, an), prepositions (to, of, for, in, on, with), common verbs (is, are, was, were, does, do, can, should), question words (what, how, why), and filler words (please, explain, tell, me, about, this, that, it). If `meaningful_token_count < min_expected_words` (default 4), the answer is flagged WARN. Example: 'the answer is yes' -> meaningful tokens = {yes} -> count = 1, flagged. 'The capital of France is Paris' -> meaningful tokens = {capital, france, paris} -> count = 3, still flagged with default threshold 4. Teams can lower to 2 for yes/no evaluation sets via config.

Q: 7. What is the over-easy check detecting — give me a concrete example that would trigger it

The over-easy check detects examples where the model could answer by copying tokens from the prompt, requiring no retrieval or reasoning. Trigger example: Prompt: 'The company was founded in 1994 by Jeff Bezos in Bellevue. When was it founded?' Answer: 'founded 1994 bellevue company jeff bezos' `prompt_tokens = {company, founded, 1994, jeff, bezos, bellevue}` `answer_tokens = {founded, 1994, bellevue, company, jeff, bezos}` Jaccard = $6/6 = 1.0$ -> WARN. This answer is literally the prompt tokens reordered. No reasoning required — a keyword extractor scores 100% on this example.

Q: 8. How is this different from just running deduplication on the dataset?

Standard deduplication removes exact duplicate rows. GoldenSetAuditor does much more: (1) Detects near-duplicates — paraphrased prompts that deduplication misses. (2) Detects conflicting labels — same prompt with different answers, which deduplication would either silently drop one or keep both arbitrarily. (3) Audits content quality — weak answers, ambiguous prompts, over-easy examples are invisible to deduplication. (4) Checks category coverage — a deduplicated set can still have catastrophically imbalanced categories. (5) Produces structured evidence — not just 'n rows removed' but 'row 47 conflicts with row 23 — here are both prompts and both expected answers.'

Q: 9. How does GoldenSetAuditor connect to DevPulse — when would you run both?

GoldenSetAuditor and DevPulse address different parts of the LLM evaluation lifecycle. DevPulse handles retrieval versioning — it detects when a knowledge base update causes previously-correct retrievals to conflict. GoldenSetAuditor audits the evaluation dataset used to measure retrieval quality. Run both in sequence: (1) update knowledge base, (2) run DevPulse to check for retrieval conflicts introduced by the update, (3) run GoldenSetAuditor to verify the golden set used to score the updated system is clean. DevPulse answers 'did my knowledge base update break anything?'; GoldenSetAuditor answers 'is the benchmark I use to detect breakage trustworthy?'

Q: 10. What does a PASS result actually mean — what is the audit NOT guaranteeing?

PASS means: no conflicting labels, no exact duplicates, no weak expected answers, no ambiguous prompts, no over-easy examples, no near-duplicate pairs, adequate category coverage. What PASS does NOT guarantee: (1) Expected answers are factually correct — only domain expert review can verify this. (2) The golden set covers the production query distribution — distribution shift is undetectable. (3) The evaluation metric is appropriate — ROUGE vs exact match vs LLM-judge is a separate decision. (4) No pretraining contamination — requires access to the model's training corpus. (5) No semantic near-duplicates — token Jaccard misses paraphrases that are surface-form different. PASS is a necessary condition for trusting a golden set, not a sufficient one.

6.3 Testing, Limitations, and Honest Positioning (Q11–Q15)

Q: 11. You have 29 tests — walk me through the most important 5

(1) `test_conflicting_labels_triggers_fail` — two rows with same prompt, different expected answers; verifies overall status = FAIL and finding is at correct row IDs. (2) `test_exact_duplicate_detection` — three rows where two are identical; verifies WARN fires and correct row pair identified. (3) `test_near_duplicate_threshold` — two prompts with controlled Jaccard just above and just below threshold; verifies the boundary behaves correctly. (4) `test_clean_golden_set_passes` — a 20-row set with no issues; verifies PASS is returned and no spurious WARNs are generated (false positive regression test). (5) `test_category_coverage_insufficient_input` — calls audit without `category_col`; verifies INSUFFICIENT_INPUT not PASS is returned for coverage check.

Q: 12. What is missing from GoldenSetAuditor that a production LLM eval team would need?

Honest answer — several things. (1) LLM-judge integration: automated assessment of expected answer factual accuracy. Currently [FUTURE]. (2) Production query distribution coverage: checking whether golden set prompts are representative of real production queries. Requires a production query log. (3) Semantic near-duplicate detection: token Jaccard misses paraphrases. (4) Multilingual support: all checks assume English stopwords and Latin-script tokenisation. (5) Version-to-version diffing: when the golden set is updated, show which new prompts are near-duplicates of existing prompts. (6) Integration with eval harnesses (HELM, lm-evaluation-harness): currently standalone.

Q: 13. How would you handle multi-turn conversation evaluation sets?

Partially supported. The conflicting labels, exact duplicate, and weak answer checks operate on the final-turn expected answer — these work correctly. Near-duplicate detection works on the full conversation hash. The ambiguous prompt check is the problematic one: anaphoric pronouns are common in multi-turn conversations ('What did it say about that?' is legitimate given prior context) and produce false positives. For multi-turn datasets: disable or raise the anaphora threshold via config. Full multi-turn support would require context-aware ambiguity detection — flagging prompts where the referent is not established in the conversation history, not just the final turn.

Q: 14. What is the difference between GoldenSetAuditor and HELM or LM-Eval harness?

HELM and lm-evaluation-harness are benchmark execution frameworks: they run a model against a fixed evaluation set and produce scores. They assume the evaluation set is correct. GoldenSetAuditor is a pre-benchmark gate: it runs before you use HELM or lm-eval, auditing whether the evaluation set is trustworthy enough to use. The tools are complementary: run GoldenSetAuditor to audit the golden set, then run HELM to benchmark the model. GoldenSetAuditor does not score model outputs; HELM/lm-eval do not audit dataset quality.

Q: 15. Why publish to PyPI? What was the design philosophy around zero-dependency core?

PyPI publication was a deliberate usability decision. Evaluation quality tooling only works if it is actually used. Friction kills adoption — if teams need to clone a repo, configure a virtual environment, and figure out imports, most will not bother. 'pip install goldensetauditor; from goldensetauditor import audit_golden_set' is the three-line path to value. The dependency philosophy: the core auditing logic uses only pandas (already in every ML environment) and Python stdlib. This means GoldenSetAuditor installs without dependency conflicts in any existing ML project. Optional features require optional installs. Principle: never make the tool harder to adopt than the problem it solves.

6.4 Deep Dives — Status Hierarchy, Error Handling, Edge Cases (Q16–Q22)

Q: 16. Walk me through the status priority hierarchy and why it is ordered that way.

FAIL > WARN > INSUFFICIENT_INPUT > PASS. FAIL means structurally broken (conflicting labels). WARN means reduced informativeness requiring human review. INSUFFICIENT_INPUT means the audit could not run completely due to missing config or large n. PASS means all checks ran and found nothing. INSUFFICIENT_INPUT sits above PASS because a golden set where some checks could not run is not the same as one that passed all checks. A report with INSUFFICIENT_INPUT should prompt the caller to add category_col to config or acknowledge the sampling limitation — not assume the set is clean.

Q: 17. How would you add a new check to GoldenSetAuditor without breaking existing users?

(1) Implement the check as a pure function check_new_thing(df, config) -> GoldenSetFinding list. (2) Add the config parameter to GoldenSetAuditConfig with a sensible default that produces INSUFFICIENT_INPUT if not provided, not a spurious WARN. (3) Add the check to audit_golden_set()'s check list. (4) Write at least one positive test (finding fires), one negative test (clean data does not fire), and one edge case. (5) Document the new finding type. Key constraint: new checks must not produce FAIL unless they detect structural invalidity equivalent to conflicting labels.

Q: 18. What is the minimum viable golden set that would pass all checks?

(1) No prompt appears twice with different answers. (2) No prompt+answer pair is exact-duplicated. (3) All expected answers have ≥ 4 meaningful tokens after stopword removal. (4) No prompts contain unresolved pronouns in < 50 tokens or multiple question marks. (5) No prompt-answer Jaccard ≥ 0.72 . (6) All prompt pairs have Jaccard < 0.82 . (7) If `category_col` is provided, each category has ≥ 3 examples. A carefully hand-curated 20-example set would pass. That does not mean it is a good golden set — it means it passes the automated structural checks.

Q: 19. How does the answer diversity check interact with the weak answer check?

They are complementary and check different failure modes. The weak answer check operates per-row: each expected answer is individually assessed for meaningful token count. The diversity check operates over the entire answer distribution: even if every individual answer passes the weak answer threshold, the answers collectively might all use the same 10 words (low TTR) or have near-zero entropy. Example: a golden set where every expected answer is 'The answer is [entity name]' might pass the weak answer check (3+ meaningful tokens each) but fail diversity (TTR near zero) — indicating templated, informationally homogeneous reference answers.

Q: 20. What error does `audit_golden_set()` raise if the `input_col` is missing from the DataFrame?

It raises a `ValueError` identifying which required column is absent: 'Column question not found in DataFrame. Available columns: [...]'. This is caught before any check runs — validation fails fast. If `expected_col` is missing, same error. If `category_col` is in config but absent from the DataFrame, the coverage check returns `INSUFFICIENT_INPUT` rather than raising — because `category_col` is optional config, its absence is a data completeness issue, not a programming error.

Q: 21. How would you benchmark `GoldenSetAuditor` itself — how do you know it has a low false positive rate?

The current test suite has 29 unit tests including explicit negative tests. What is missing is an empirical false positive rate against real-world golden sets. To measure this properly: (1) Collect 10+ real golden sets from open benchmarks (MMLU, TriviaQA, HellaSwag subsets). (2) Run `GoldenSetAuditor` on each. (3) For every `WARN` finding, have a human expert label it: true positive (genuine quality issue) or false positive (legitimate pattern mis-flagged). (4) Compute precision = $TP / (TP + FP)$ per check type. This has not been done — it is a known gap in the validation methodology.

Q: 22. How do you handle missing values in the expected_answer column?

Rows where `expected_col` is NaN or empty string are treated as weak expected answers (0 meaningful tokens after stopword removal) and flagged by the weak answer check. Other checks handle NaN/empty defensively: token comparison functions normalise None/NaN to empty string before tokenising so they do not raise `TypeError`. The conflicting labels check would flag rows where two prompts match but one has NaN expected and one has a real answer — which is correct, that is a genuine data quality issue.

6.5 Advanced Topics — Scalability, Roadmap, CI Integration (Q23–Q29)

Q: 23. Explain the category coverage check's INSUFFICIENT_INPUT vs WARN distinction.

`INSUFFICIENT_INPUT` fires when `category_col` is not provided in config — the check could not run at all because the required information was not supplied. `WARN` fires when `category_col` is provided, the check ran, and at least one category has fewer than `min_category_count` examples. The distinction matters for report consumers: `INSUFFICIENT_INPUT` tells you to add `category_col` to config; `WARN` tells you to add more examples to underpopulated categories. `INSUFFICIENT_INPUT` = 'we do not know if coverage is adequate'; `WARN` = 'we know coverage is inadequate for these specific categories'.

Q: 24. Walk me through the most likely way GoldenSetAuditor would produce a wrong result.

The most likely wrong result is a false `PASS` on the near-duplicate scan when $n > 5,000$ and the sampling misses true near-duplicate pairs. With 25,000 sampled pairs from a 10,000-row dataset (49.9M total pairs), the sampling fraction is 0.05%. A golden set with 50 near-duplicate pairs would have expected 2.5 pairs in the sample — possible to miss. The fix: increase `max_pairwise_checks` or switch to embedding-based approximate nearest neighbor search (FAISS) which is $O(n \log n)$. This is documented as a known limitation on the `INSUFFICIENT_INPUT` finding when sampling is used.

Q: 25. What is the computational profile of running all 7 checks on a 1,000-row dataset?

Conflicting labels: $O(n)$ hash map $\rightarrow$ $<5\text{ms}$. Exact duplicates: $O(n)$ $\rightarrow$ $<5\text{ms}$. Weak answer check: $O(n)$ with token set construction $\rightarrow$ $\sim 50\text{ms}$. Ambiguous prompt: $O(n)$ regex $\rightarrow$ $\sim 100\text{ms}$. Over-easy check: $O(n)$ token Jaccard(prompt, answer) $\rightarrow$ $\sim 50\text{ms}$. Near-dup scan: $O(n^2) = 499,500$ pairs $\rightarrow$ $\sim 3\text{-}5$ seconds. Category coverage: $O(n)$ groupby $\rightarrow$ $<5\text{ms}$. Total: $\sim 4\text{-}6$ seconds, dominated by near-dup scan. Parallelisation is possible for near-dup (each pair comparison is independent) but threading overhead is not worth it below $n=5,000$.

Q: 26. Why is stopwords removal necessary before token counting? What happens without it?

Without stopwords removal, 'The answer is yes' has 4 tokens and passes `min_expected_words=4`. With stopwords removal, {the, is} are removed, leaving {answer, yes} — 2 meaningful tokens, flagged. For Jaccard similarity: stopwords add noise that depresses similarity scores. 'What is the capital of France?' vs 'What is France capital?' share 'what', 'is', 'the', 'of' — stopwords that inflate the intersection artificially. Removing them makes Jaccard reflect content word overlap, which is what we care about for near-duplicate detection.

Q: 27. How does the audit report get attached to a model card or evaluation documentation?

`report.save('outputs/')` generates three files: JSON, Markdown, HTML. The Markdown output is designed to be embedded directly in a model card (GitHub README, HuggingFace model card) — readable as plain text and renders correctly. The JSON output is machine-readable and can be parsed by downstream tooling to fail a CI check if `overall_status != 'PASS'`. The HTML is a self-contained report for human reviewers. Typical workflow: run in CI, save to artifacts, attach JSON to model card, fail CI if `status == 'FAIL'`.

Q: 28. If you had to cut to 3 checks instead of 7, which 3 would you keep?

(1) Conflicting labels — the only FAIL check; structural invalidity that must be fixed. (2) Near-duplicate scan — the most common real-world finding; manually assembled golden sets almost always contain paraphrased duplicates that inflate apparent diversity. (3) Category coverage — the easiest-to-miss failure mode; a golden set with 2 examples in a category produces statistically meaningless per-category scores that teams routinely over-interpret. The other four checks improve evaluation quality but are less likely to fundamentally invalidate results. The three keepers address logical invalidity, phantom diversity, and phantom coverage.

Q: 29. What would a v1.0 of GoldenSetAuditor look like?

v1.0 would add: (1) LLM-judge expected answer verification — for each expected answer, call an LLM judge with the prompt and ask whether the answer is factually accurate, returning a confidence score. (2) Semantic near-duplicate detection via sentence-transformers + FAISS. (3) Production distribution coverage check — given a sample of production queries, compute embedding-based distance between production queries and golden set prompts to detect distribution gaps. (4) Version-to-version diffing — when a new golden set version is provided, show what changed and whether new prompts are near-duplicates of existing ones. (5) Integration hooks for HELM and lm-evaluation-harness. These would require moving beyond zero-runtime-dependency, which is an acceptable tradeoff for a v1.0 production tool.

6.6 Stress Test Questions — Adversarial, Output Formats, CI Workflow (Q30–Q32)

Q: 30. How does GoldenSetAuditor's design handle adversarial misuse — can a bad actor craft a golden set that passes all checks but is still problematic?

Yes, and this is a known limitation documented in every report. An adversary who knows the check logic can craft: (1) Near-duplicates just below the 0.82 Jaccard threshold — e.g., Jaccard=0.80 pairs that are effectively the same question phrased identically but with one substituted content word. (2) Expected answers that are long (pass weak-answer check) but factually wrong. (3) Prompts with one question mark that are genuinely ambiguous. GoldenSetAuditor is a quality floor, not a quality ceiling. It catches the mechanical, observable failure modes that plague naturally assembled golden sets. Adversarial failure is a different threat model requiring human domain expert review on top of the automated gate. This is why the truth boundary is non-suppressible.

Q: 31. Explain how the GoldenSetReport.save() method produces three output formats from one object.

GoldenSetReport.save(output_dir) dispatches to three format-specific serializers: (1) JSON: report.__dict__ is serialised recursively; each GoldenSetFinding becomes a JSON object with check_name, status, row_ids, message, evidence. Machine-readable for CI integration — a pipeline step can read overall_status and fail the build if FAIL. (2) Markdown: findings are formatted as a table with check name, status, and message; the truth boundary appears as a blockquote. Designed to be embedded in GitHub README or HuggingFace model card as-is. (3) HTML: a self-contained report with inline CSS; status badges use color coding (red/amber/green); evidence blocks are expandable table rows. All three are written to output_dir in one call. Format choice is at consumption time, not at audit time.

Q: 32. How would you integrate GoldenSetAuditor into a CI/CD pipeline for a team that updates their golden set monthly?

The integration is a single CI step: (1) On every PR that touches the golden set file, add a CI job that runs: `pip install goldensetauditor; python run_audit.py`. (2) `run_audit.py` calls `audit_golden_set(df, config)`, saves the report to artifacts, and exits with code 1 if `overall_status == 'FAIL'`. CI blocks the merge on FAIL. (3) For WARN: the PR can merge but the CI artifact shows the warnings — the team reviews them in the next sprint, not blocking delivery. (4) Report JSON is attached to the PR as a comment (via GitHub Actions) so reviewers see the audit summary without downloading artifacts. (5) Over time, track the WARN history: if the same check fires on every golden set update, the threshold may need domain-specific calibration. This workflow adds less than 30 seconds to PR CI for sets under 1,000 rows.

S7 Failure Modes

Failure Mode	Trigger	Impact	Mitigation
Near-dup scan misses pairs at $n > 5K$	Random sampling 25K pairs from 49M total	False PASS on near-dup check	Increase max_pairwise_checks or use embedding ANN (FAISS)
Ambiguous prompt false positives	Legitimate anaphora in multi-turn sets	False WARN; teams distrust the check	Disable anaphora check for multi-turn datasets via config flag
Weak answer check on domain short answers	Medical yes/no QA; correct answers are 1-2 tokens	False WARN on every expected answer	Lower min_expected_words to 1 in GoldenSetAuditConfig
Over-easy false positives	Definition prompts that legitimately repeat answer terms	False WARN; flagging valid examples	Raise overeasiness_threshold to 0.85 for definitional sets
Conflicting labels miss unicode variants	Unicode variation, RTL text, accented characters	True conflicts not detected	Add NFKC normalisation to normalise() before comparison
Coverage PASS misread when no category_col	No category_col -> INSUFFICIENT_INPUT returned	Teams may interpret as clean; skip adding category data	INSUFFICIENT_INPUT status prominently documented; CI fails on it

7.2 Failure Mode Severity Classification

Not all failure modes have equal impact on evaluation validity:

Critical (blocks trust in evaluation): Near-dup scan misses at large n . If a large golden set has many near-duplicate pairs and the sampling misses them, every benchmark number produced by that golden set is inflated. The fix is to use embedding-based ANN search (FAISS) for $n > 5,000$.

Moderate (introduces noise, recoverable): Ambiguous prompt false positives and weak answer false positives. These produce WARN, not FAIL. They require human review time but do not silently corrupt results — the flagging makes the issue visible.

Low (edge cases, specific domains): Unicode normalisation gaps affect only multilingual golden sets. Coverage PASS misread requires caller to not read the INSUFFICIENT_INPUT status — unlikely in a well-integrated CI pipeline.

S8 Scalability

GoldenSetAuditor is designed for evaluation sets in the range of 100 to 5,000 examples, covering the large majority of RAG and LLM evaluation workloads. The near-duplicate scan's $O(n^2)$ complexity is the primary constraint.

Check	Complexity	100 rows	1,000 rows	10,000 rows
Conflicting labels	$O(n)$	<1ms	<5ms	<50ms
Exact duplicates	$O(n)$	<1ms	<5ms	<50ms
Weak answer check	$O(n)$	<5ms	<20ms	<200ms
Ambiguous prompt	$O(n)$	<10ms	<50ms	<500ms
Over-easy detection	$O(n)$	<5ms	<20ms	<200ms
Near-dup scan	$O(n^2)$	<1s	~3-5s	INSUFFICIENT_INPUT (sampled)
Category coverage	$O(n)$	<1ms	<5ms	<50ms

Scalability Note: The near-dup scan's $O(n^2)$ growth is the primary constraint. For 100-1,000 examples (the dominant range for RAG evaluation sets), all checks run in under 10 seconds. For sets above 5,000, the auditor returns INSUFFICIENT_INPUT on near-dup and documents the sampling fraction. This is an explicit, honest trade-off.

8.2 Scaling Strategy for Large Golden Sets (> 5,000 rows)

For production golden sets above the sampling threshold, the recommended architecture is:

Step 1 — Embedding-based ANN pre-filter: Encode all prompts with a lightweight sentence-transformer (e.g., all-MiniLM-L6-v2, 22M parameters). Build a FAISS index. For each prompt, retrieve its 10 nearest neighbors. This is $O(n \log n)$ and scales to 100K+ examples.

Step 2 — Token Jaccard confirmation: For each candidate near-duplicate pair returned by ANN, compute token Jaccard as confirmation. This two-stage approach reduces false positives from ANN (which can return semantically similar but actually distinct prompts) while keeping complexity sub-quadratic.

Step 3 — Batch all $O(n)$ checks: Conflicting labels, exact duplicates, weak answer, ambiguous prompt, over-easy, and category coverage are all $O(n)$ and scale trivially. Parallelise with Python multiprocessing if needed.

Step 4 — Incremental auditing: For monthly golden set updates, only audit new and modified rows for $O(n)$ checks; only compute ANN distances for new prompts against the existing index. Full re-audit is unnecessary if the index is maintained.

8.3 Memory Profile

For $n=1,000$ rows with average 20-token prompts: token sets are ~200 bytes each. The pairwise comparison matrix for near-dup ($n*(n-1)/2 = 499,500$ pairs) processes in streaming fashion — no full matrix held in memory. Peak memory for $n=1,000$ is approximately 50MB including the DataFrame. For $n=10,000$ with sampling: the 25,000 sampled pairs are processed in batches of 1,000, peak memory

stays under 100MB.

S9 Cross-Project Connections

GoldenSetAuditor is the evaluation dataset quality gate in a 13-repo Applied LLM Systems portfolio. Understanding how it connects to other projects demonstrates system-level thinking.

Project	Connection to GoldenSetAuditor
DevPulse	DevPulse detects retrieval conflicts when a knowledge base changes. GoldenSetAuditor audits the golden set used to measure retrieval quality. Run GoldenSetAuditor first (gate golden set quality), then DevPulse (gate retrieval quality after KB update). DevPulse answers 'did my KB update break retrievals?'; GoldenSetAuditor answers 'is my benchmark trustworthy?'
MetaSignal	MetaSignal runs A/B experiments to compare model versions using a primary metric. If that metric is based on golden set accuracy, GoldenSetAuditor must gate the golden set before MetaSignal experiment results are trusted. A near-duplicate golden set inflates accuracy, making one model appear better than another on phantom diversity.
DocIngestQA	DocIngestQA audits RAG document ingestion quality (chunking, embedding, retrieval). GoldenSetAuditor audits the evaluation dataset used to score that ingestion pipeline. Both are quality gates at different layers: ingest quality and evaluation dataset quality. Run both before reporting RAG system benchmarks.
TrialCheck	TrialCheck audits A/B experiment readouts for statistical validity. If an experiment measures model improvement on a golden set, TrialCheck validates the experiment design (SRM, peeking, underpowered); GoldenSetAuditor validates the evaluation dataset used in the experiment. Neither replaces the other: a statistically valid experiment on a corrupt golden set is meaningless.
FeatureLeakageLens	FeatureLeakageLens detects pre-training contamination between training data and evaluation sets. GoldenSetAuditor detects surface-form near-duplicates within the golden set. FeatureLeakageLens detects overlap between the golden set and the model's training corpus. Both are contamination signals; GoldenSetAuditor is internal quality; FeatureLeakageLens is external contamination.

9.2 The Full Quality Pipeline — Where Each Tool Runs

A complete, defensible LLM evaluation pipeline has quality gates at every stage:

Stage 1 — Golden set quality: GoldenSetAuditor. Before any benchmark runs. Gate: overall_status must not be FAIL.

Stage 2 — Retrieval quality (RAG systems): DocIngestQA + DevPulse. After knowledge base update. Gate: no retrieval conflicts introduced.

Stage 3 — Experiment design: TrialCheck. Before reading out A/B results. Gate: no SRM, no peeking, adequate sample size.

Stage 4 — Model scoring: HELM / lm-evaluation-harness. Runs against the gate-passed golden set. Output: model accuracy, latency, cost.

Stage 5 — Version comparison: MetaSignal. A/B test between two model versions, readout gated by TrialCheck.

GoldenSetAuditor is Stage 1. It must pass before any other stage's results are meaningful. Skipping Stage 1 is like running a clinical trial without verifying the measurement instrument.

9.3 Answering 'How Does This Fit into a Production System?' at FAANG

A FAANG interviewer asking about system-level fit is evaluating whether you think like a systems engineer, not just a tool author. The expected answer covers four dimensions:

- 1. Trigger:** GoldenSetAuditor runs on every change to the golden set file. This is a CI gate, not a scheduled batch. The trigger is a git push or PR that modifies the golden set CSV/JSON — detected by file-path filter in the CI config.
- 2. Integration point:** As a pytest fixture or standalone CI step. The output JSON is written to CI artifacts. The exit code communicates FAIL/PASS to the CI system. A GitHub Action can comment the findings summary on the PR.
- 3. Ownership:** The team that owns the golden set owns the audit results. GoldenSetAuditor does not send alerts or file tickets — it produces evidence. The PR author reviews the findings before merging. FAIL blocks merge; WARN requires explicit acknowledgment (approve despite warnings).
- 4. Observability:** Over time, track which checks fire most frequently. If near-duplicate warnings appear on every golden set update, the threshold may be miscalibrated for that domain. If conflicting labels never fire, the annotation process is disciplined. Audit finding history is a signal about annotation process quality, not just individual golden set quality.

This framing — CI gate, artifact output, team ownership, observability over time — is what distinguishes a systems-thinking answer from a tool-description answer.

S10 Resume Claim Audit

Resume-safe claim (from README):

Built GoldenSetAuditor, an evaluation dataset quality auditor for LLM/RAG applications that checks golden sets for conflicting expected answers, exact and near-duplicate prompts, weak reference answers, ambiguous questions, over-easy examples, and category coverage gaps, producing structured JSON/Markdown/HTML audit reports with per-finding PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary. Published to PyPI. 29 tests.

Claim	Defensible?	Evidence
conflicting expected answers	YES	check_conflicting_labels in core.py — FAIL status
exact and near-duplicate prompts	YES	check_exact_duplicates + pairwise token Jaccard $O(n^2)$
weak reference answers	YES	STOPWORDS set + meaningful token count < min_expected_words
ambiguous questions	YES	anaphora regex + multiple question mark heuristic
over-easy examples	YES	Jaccard(prompt_tokens, answer_tokens) >= 0.72 threshold
category coverage gaps	YES	per-category count vs min_category_count threshold
JSON/Markdown/HTML reports	YES	GoldenSetReport.save(output_dir) — three format files
PASS/WARN/FAIL/INSUFFICIENT_INPUT	YES	four-level priority hierarchy in aggregate_status()
explicit truth boundary	YES	non-suppressible text in JSON, MD, and HTML outputs
Published to PyPI	YES	pip install goldensetauditor installs and runs
29 tests	YES	tests/ directory — 29 pytest cases pass in CI

10.2 What You Can NOT Claim From This Project

Do NOT claim: 'I built a system that validates LLM outputs' or 'I built an evaluation framework for LLM models.' GoldenSetAuditor audits the EVALUATION DATASET, not the model. If asked 'does this tell you if the model is good?', the correct answer is: 'No. It tells you if the benchmark you use to score the model is trustworthy. Those are different questions and this tool answers only the first.'

10.3 Resume Bullet Options

Senior Engineer (1-line bullet): Built GoldenSetAuditor — a PyPI-published evaluation dataset quality auditor for LLM/RAG systems implementing 7 deterministic checks (token Jaccard near-duplicate scan, conflicting label detection, Shannon entropy diversity, category coverage), generating JSON/Markdown/HTML audit reports with per-finding evidence. 29 tests.

Mid-level (2-line version): Built GoldenSetAuditor to address the gap in automated quality assurance for LLM evaluation datasets — the step that happens before HELM or lm-evaluation-harness runs. Implements 7 checks including $O(n^2)$ pairwise token Jaccard near-duplicate scan with transparent sampling above 5,000 rows. PyPI published. Connects to TrialCheck, DevPulse, and MetaSignal as part of a 13-project portfolio.

10.4 What the Interviewer Is Actually Evaluating

Understanding the evaluation criteria makes preparation more targeted. For a senior engineering role at FAANG, a project discussion tests five dimensions:

Problem decomposition: Can you articulate the precise problem (golden set quality gap) and why it matters (downstream model selection decisions) without prompting? S1 covers this. If you can explain the problem before the interviewer asks, you demonstrate product thinking.

Algorithm design: Can you derive and explain the core algorithms (token Jaccard, Shannon entropy, normalisation pipeline) from first principles? S3 covers this. The interviewer is not testing memory — they want to see you reason through the math and connect it to the problem.

Design trade-offs: Can you articulate why each design decision was made and what alternative was rejected? S4 covers this. 'I chose token Jaccard because X and rejected embedding similarity because Y' is the format for every answer.

Honesty about limitations: Can you proactively identify what the tool does not do and why? S5 and S7 cover this. Admitting limitations before being challenged signals engineering maturity, not weakness.

Systems thinking: Can you place the tool in a larger engineering context? S9 covers this. The question 'where would this fit in a production system?' requires knowing GoldenSetAuditor's role relative to HELM, TrialCheck, DevPulse, and CI/CD infrastructure.

The single most common failure mode in project deep-dive interviews: answering WHAT instead of WHY. 'The check computes token Jaccard' is WHAT. 'Token Jaccard was chosen because it requires zero ML dependencies, is deterministic, and catches surface-form near-duplicates which are the dominant failure mode in manually assembled golden sets' is WHY. Always lead with WHY.

S11 Study Plan + Do Not Say This

11.1 One-Week Study Plan

Day 1: Read core.py start to finish. Trace audit_golden_set() through every check. Draw the call graph on paper.

Day 2: Work through S3 (Algorithm Explanations) manually. Compute token Jaccard by hand for 3 prompt pairs. Compute Shannon entropy for a 5-answer set.

Day 3: Run the demo: pip install goldensetauditor; generate a report. Read the JSON output. Modify one row to create a conflicting label. Verify FAIL fires.

Day 4: Practice Q&A; pairs from S6. Focus on the required 15. For each answer, identify the key principle being tested.

Day 5: Study the design decisions in S4. For each, argue the alternative without notes — the interviewer may push back.

Day 6: Read the scalability section. Practice explaining $O(n^2)$ and why sampling is the right tradeoff at large n .

Day 7: Mock interview. Use S6 as a script. Time yourself. Stop using notes after the first pass.

11.1a What to Do if the Interviewer Asks Something Not in This Document

Even with 32 Q&A; pairs, an interviewer can ask something unexpected. The fallback strategy:

First principle: always know what the tool DOES NOT do. If asked about something outside the scope of the seven checks, the clean answer is: 'That is out of scope for GoldenSetAuditor — it audits dataset quality, not [X]. Here is why [X] is a separate problem and what tool would address it.'

Second principle: connect to the design decisions. Almost any question about implementation choices can be answered by referencing one of the 8 design decisions in S4: FAIL boundary, token Jaccard choice, INSUFFICIENT_INPUT status, truth boundary, PyPI publication, configurable thresholds, four-level hierarchy, structured evidence.

Third principle: admit known gaps honestly. 'That is a known limitation — here is why we chose not to address it in v0.5.0 and here is how v1.0 would fix it.' This is stronger than a vague claim that the tool handles everything.

Fourth principle: derive, do not recall. If you can derive token Jaccard from scratch ('I need a set-based similarity measure that works on tokens, is symmetric, bounded 0-1, and requires no external dependencies — that is Jaccard'), you can handle any variant the interviewer proposes ('what if you weighted by IDF?').

11.1b Interview Timing Guidance

Target answer lengths: System design walk-through (Q1) — 3-4 minutes. Algorithm questions (Q3-Q7) — 2-3 minutes, spend 45 seconds on the formula derivation, 45 seconds on the worked example. Design decision questions (S4) — 2 minutes: state the decision, state the alternative, explain why the alternative was rejected. Limitation questions — 60-90 seconds: name it, quantify it if possible, explain the fix. Never say 'I don't know' — say 'That is not currently implemented; here is why and here is the roadmap.'

11.2 Common Interview Ambush Questions — Pre-Prepared Answers

These questions sound hostile but have clean answers if you know the project:

'Isn't this just pandas deduplication?' — No. Standard deduplication removes exact rows. GoldenSetAuditor detects conflicting labels (same prompt, different answers), near-duplicates (paraphrased prompts), content quality issues (weak answers, ambiguous prompts, over-easy examples), and category coverage gaps. Deduplication cannot do any of these.

'Your token Jaccard is too simple for real-world use.' — Token Jaccard is deliberately simple: zero ML dependencies, deterministic, explainable. For most manually assembled golden sets, surface-form near-duplicates are the dominant failure mode. Semantic near-dup via sentence-transformers is on the roadmap for v1.0.

'Did you test this on real golden sets?' — 29 unit tests covering positive, negative, and edge-case scenarios. Empirical false positive rate measurement on open benchmark golden sets is listed as a known gap — documented, not hidden.

'What is the scalability story?' — All checks except near-duplicate are $O(n)$. Near-dup is $O(n^2)$; above 5,000 rows it samples and returns `INSUFFICIENT_INPUT`. For >5,000 rows the recommended upgrade is FAISS ANN, reducing complexity to $O(n \log n)$.

11.3 Do Not Say This

NEVER SAY THIS	SAY THIS INSTEAD
"It detects duplicates in the dataset"	"It detects conflicting labels — same prompt, different answers — and near-duplicate prompts that inflate apparent diversity without adding evaluation coverage."
"It validates the model outputs"	"It audits the evaluation dataset. It explicitly does NOT evaluate model outputs — the truth boundary is non-suppressible in every report format."
"The checks run with ML models"	"All checks are deterministic heuristics — token Jaccard, stopword filtering, regex-based anaphora detection. No ML model involved. Deliberate choice for reproducibility and zero-dependency deployment."
"It scales to millions of examples"	"Designed for 100-5,000 examples. Near-dup scan is $O(n^2)$; above 5,000 it samples and returns <code>INSUFFICIENT_INPUT</code> . For larger sets I would use embedding-based ANN."
"WARN means the set is bad"	"WARN means a quality issue was detected that requires human review. Whether it matters for your specific domain is a judgment call. Only FAIL requires fixing."
"I used BERT or GPT for the checks"	"Zero external ML dependencies. All checks are pure Python plus pandas plus numpy."

11.4 The 30-Second Elevator Pitch for GoldenSetAuditor

Every project needs a 30-second version that can be delivered naturally when the interviewer says 'tell me about a project you worked on.' Memorise this:

GoldenSetAuditor is a PyPI-published tool that audits evaluation datasets for LLM and RAG systems before benchmarking runs. It addresses the problem that teams spend significant effort improving models but rarely validate the quality of the golden set used to measure improvement. The tool runs 7 deterministic checks — including pairwise near-duplicate detection using token Jaccard, conflicting label detection, and Shannon entropy diversity analysis — and produces structured JSON, Markdown, and HTML reports with per-finding evidence. It is designed as a CI gate: on every golden set update, the CI checks that no structural quality issues (conflicting labels) exist before allowing the golden set to be used for model evaluation. 29 tests. Zero external ML dependencies. `pip install goldensetauditor`.

11.5 Quick-Reference Formula Card

These formulas appear in interview questions. Know them cold:

Formula	What It Computes	Used In
$J(A,B) = A \cap B / A \cup B $	Token Jaccard similarity between two token sets	Near-dup scan, over-easy check, answer diversity
$H = -\sum p(t) \cdot \log_2(p(t))$	Shannon entropy of token frequency distribution	Answer diversity check (entropy component)
$H_{\text{norm}} = H / \log_2(\text{vocab_size})$	Normalised entropy, range [0,1]	diversity_entropy_warn threshold comparison
$\text{TTR} = \text{unique_tokens} / \text{total_tokens} $	Type-token ratio — vocabulary reuse rate	Answer diversity check (TTR component)
$\text{CV} = \text{std}(\text{lengths}) / \text{mean}(\text{lengths})$	Coefficient of variation of answer lengths	Answer diversity check (length variation)
$\text{pairs} = n \cdot (n-1) / 2$	Number of pairwise comparisons for near-dup scan	Complexity analysis: $O(n^2)$ justification
<code>normalise(t) = lower + strip + collapse_spaces + strip_boundary_punct</code>	Text normalisation for conflicting label matching	Conflicting labels check

GoldenSetAuditor Interview Defense v3.0 | Sidharth Kriplani | 32 Q&A; Pairs | FAANG Standard
github.com/SidharthKriplani/goldensetauditor | `pip install goldensetauditor`

SA Glossary of Terms Used in This Document

Term	Definition as Used in GoldenSetAuditor Context
Golden set	A collection of (prompt, expected_answer) pairs used as ground truth for LLM/RAG evaluation. Also called evaluation dataset, benchmark set, or test set.
Token Jaccard	Set-based similarity metric: $J(A,B) = A \cap B / A \cup B $ where A and B are sets of non-stopword tokens extracted from two text strings.
STOPWORDS	Fixed set of English function words removed before tokenisation. Includes articles, prepositions, common verbs, question words, and fillers. Not configurable in v0.5.0.
Conflicting labels	Two or more rows in the golden set with identical normalised prompts but different expected answers. The only FAIL-level finding.
Near-duplicate	Two prompts with token Jaccard $\geq$ near_duplicate_threshold (default 0.82). Indicates paraphrased versions of the same question that inflate apparent diversity.
Over-easy example	An (prompt, answer) pair where token Jaccard $\geq$ overeasy_overlap_threshold (default 0.72). Indicates the model could answer by copying tokens from the prompt.
Weak expected answer	An expected answer with fewer than min_expected_words (default 4) meaningful tokens after stopwords removal.
INSUFFICIENT_INPUT	Check status returned when a check could not run due to missing optional config or exceeded complexity limit ($n > 5,000$ for near-dup). Not PASS — explicitly different.
Truth boundary	Non-suppressible text printed in every report format stating what GoldenSetAuditor does and does not audit. Cannot be removed via configuration.
GoldenSetAuditConfig	Python dataclass holding all configurable parameters: column names, thresholds, sampling limits. Passed to audit_golden_set() as the second argument.
GoldenSetReport	Return value of audit_golden_set(). Contains overall_status, list of GoldenSetFinding objects, truth_boundary, and config_used.
GoldenSetFinding	A single quality issue found by one check. Contains check_name, status, row_ids, message, and evidence dict.
Shannon entropy (H)	Information-theoretic measure of token distribution diversity. $H = -\sum p(t) \cdot \log(p(t))$. Normalised by $\log(\text{vocab_size})$ for the diversity check.
Type-Token Ratio (TTR)	Unique token count / total token count. Measures vocabulary diversity. Low TTR indicates templated or repetitive expected answers.
$O(n^2)$	Quadratic time complexity. For n examples, the near-duplicate scan compares $n \cdot (n-1)/2$ pairs. Above $n=5,000$, replaced by sampling of max_pairwise_checks random pairs.